

Queen's University Belfast

School of Mathematics and Physics

The Modelling and Simulation of the Belfast Bike-Share System

MTH3024: Modelling and Simulation

Students:

Ben Hamilton	40405601
Luke Jackson	40400730
Alex O'Connor	40399054
Adam Higgins	40432789

Date: February 9, 2026

Abstract

This project develops a stochastic simulation of the Belfast Bike-Share system using observed journey data to model bike movements. A Markov framework is used to reproduce demand patterns and identify the causes of station imbalance, measured through no-bike and overflow failures. The model is validated by comparing the simulated and observed trip start and end distributions, which match. Optimisation strategies are then examined, including station-specific dock allocation and bike rebalancing, with rebalancing producing the largest reduction in unsatisfied demand. An extension using proportional symbol mapping reveals strong intra-day variation, with morning inflows toward the city centre and incomplete evening redistribution, highlighting the importance of time-dependent operational interventions.

Contents

Abstract i

1 Introduction 1

2 Methods 1
- 2.1 Data . . . 1
- 2.2 Understanding the Basic Model . . . 1
  - 2.2.1 Initial bike allocation . . . 1

3 Results and Discussion 2
- 3.1 Validation . . . 2
- 3.2 Results of the Basic Model . . . 2

4 Optimisation 3
- 4.1 Optimisation: Dock Allocation and Bike Rebalancing . . . 3
  - 4.1.1 Active bike rebalancing . . . 4
- 4.2 Optimisation Results . . . 4

5 Extension: Mapping Intra-day Changes 5
- 5.1 Model . . . 5

6 Conclusions, Limitations and Further Work 6
- 6.1 Limitations . . . 6
- 6.2 Further Work . . . 6

A Appendix: Raw Data Description 8

B Appendix: Excel Processing Workflow 8

C Appendix: Data Cleaning and Preprocessing 9

D Appendix: Construction of Flow and Transition Matrices 9

E Appendix: Stochastic Departure Model 9

F Appendix: Simulation Initialisation Details 10

G Appendix: Python Implementation 10

H Appendix: Supplementary Figures and Outputs 12

1 Introduction

Bike-share systems have become a common form of urban transport, offering flexible, low-cost mobility for short trips. Their aim is to ensure that bikes are reliably available for shared use at an affordable price.
The Belfast Bikes scheme began in April 2015, expanding from 30 to 48 docking stations and from 300 to 400 bikes between 2015 and 2021[1]. Optimising bike-share systems is recognised as important to improve urban mobility (Zhang et al 2022 [2]), supporting affordable transport provision (Qian et al 2024 [3]) and facilitating everyday commuting, including travel to and from Belfast's two major universities.
This report focuses on three key questions:

1. What problems in the current bike-share system are causing uneven pickup and return rates across stations?
2. How can the bike-share system be optimised to improve station availability and overall performance?
3. What underlying factors drive the differences in pickup and return rates across stations in the bike-share system?

2 Methods

2.1 Data

The datasets consist of recorded bike journeys from Open Data NI. Each row represents a completed journey, including its starting and destination stations.
After cleaning, journeys were converted into a flow matrix F, where F_{ij} is the total number of observed trips from station i to station j. The flow matrix was normalised to obtain the transition probability matrix P with entries

$$p_{ij} = \frac{F_{ij}}{\sum_k F_{ik}}$$

which represent the probabilities that a bike from station i moves to station j. Further details on data preparation and cleaning are provided in Appendix C.

2.2 Understanding the Basic Model

Our model is a discrete time stochastic process, with the state being the number of bikes at a station at time t. The total number of bikes in the system is fixed throughout the simulation, and each station has a finite docking capacity.

Each time step is a one minute interval, bikes may depart from stations according to estimated departure rates λ_i. However, a departure from station i can only occur if a bike is available. When a departure occurs, the destination is chosen according to the transition probability matrix P derived from observed journey data.

Bike movements update the state by decrementing the origin station and incrementing the destination station. Multiple bikes can arrive at the same station within a single time step if they arrive from different stations.

This framework induces a Markov structure, since the future evolution of the system depends only on the current distribution of bikes across stations. Given this state, together with the departure rates and transition probabilities, all subsequent movements are fully determined

2.2.1 Initial bike allocation

The starting number of bikes at each station is based on how many journeys *begin* there in the data. Stations with more departures receive more bikes at the start.
For each station i, the value

$$v_i = \frac{\sum_j F_{i,j}}{\sum_k \sum_j F_{k,j}}$$

gives the fraction of all recorded trips that start at station i.
Let N be the total number of stations, B_{tot} the total number of bikes, and $b_{\min}$ a small baseline allocation for every station. The remaining bikes are distributed using the following equation:

$$b_i = b_{\min} + \lfloor (B_{\text{tot}} - N b_{\min})\, v_i \rfloor .$$

Any bikes left over after rounding are assigned to the stations with the largest values of v_i.

[1]For details on the growth of the Belfast Bikes scheme, see `https://minutes.belfastcity.gov.uk/mgAi.aspx?ID=64023`

3 Results and Discussion

3.1 Validation

To assess the accuracy of the model, we compare the observed and simulated start and end shares for the highest-demand stations. Figure 1 illustrates this comparison for the top ten stations by total trip volume

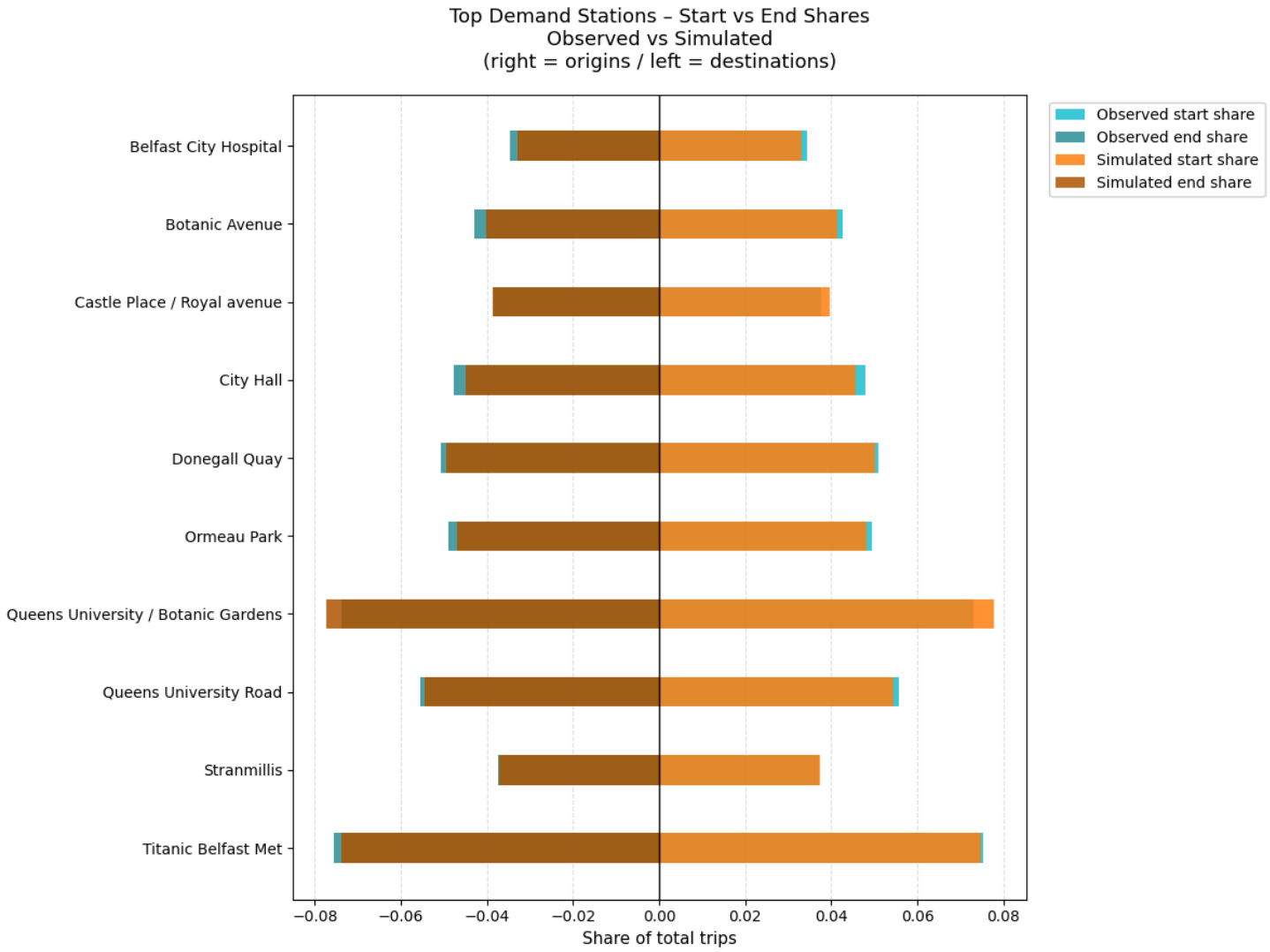


Figure 1: Top 10 demand stations as per share of total trips, showing the difference between the simulated system and the observed data.

The simulated start and end shares match the observed percentages for the top-demand stations, such as Belfast City Hospital, where the observed start share is 0.034489 and the simulated start share is 0.033159, giving an absolute difference of 0.001329 and a percentage difference of 3.854763%. This indicates that the model captures the real distribution of trip origins and destinations, supporting its validity.

3.2 Results of the Basic Model

We begin by simulating a full day to observe station behaviour, sorting stations by decreasing pick-ups to highlight large differences between pick-ups and returns.

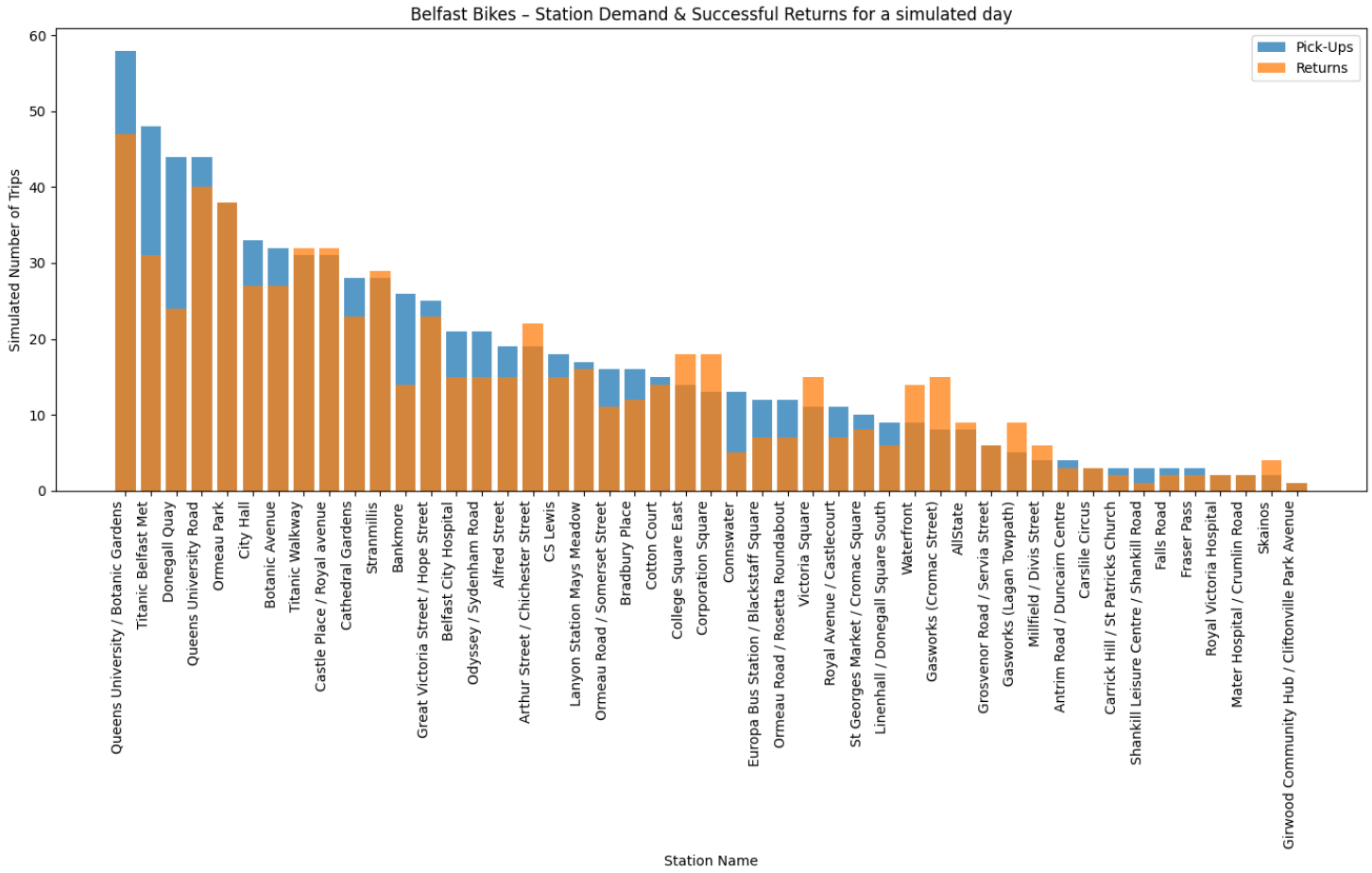


Figure 2: Simulated system for one day (1440 minutes), showing the number of Pick-ups and Returns at each station.

From this model, we will focus on question (1) about the causes of uneven pickup and return rates. We can make the following inferences:

- **High-Demand Student Stations.** Stations located near Queen's University have the highest number of pick-ups, which is consistent with the high daily demand in student areas.
- **Imbalance in Pick-Ups and Returns.** Across many stations, we observe a large difference between the number of pick-ups and returns during the day. This suggests that bikes are taken from these locations far more regularly than they are returned, indicating that riders frequently travel from these stations to other parts of the city.

  In contrast, stations such as Corporation Square show the opposite behaviour, with more returns than pick-ups. This station is close to Clarendon Dock, one of Belfast's industrial areas, so it is reasonable to infer that many commuters cycle from elsewhere to work here and then use alternative transport in the evening, leading to consistently higher return counts, answering our first research question (1).

4 Optimisation

Let D_i^{fail} be the total number of no-bike failures at station i, and let A_i^{fail} be the total number of overflow failures at station i during the simulation. We define the total cost function for unsatisfied customers as

$$J = \sum_{i=1}^{N} \left(D_i^{\text{fail}} + A_i^{\text{fail}} \right), \tag{1}$$

The goal is to choose dock capacities c_i and rebalance bikes to reduce J.

4.1 Optimisation: Dock Allocation and Bike Rebalancing

The main goal of optimisation is to reduce the number of unsatisfied users of the bike sharing system. In the model, an unsatisfied user occurs when a user attempts to start a trip from station i but $b_i(t) = 0$ (no-bike failure), or a user attempts to end a trip at station j but $b_j(t) = c_j$ (overflow failure).

4.1.1 Active bike rebalancing

We observe that asymmetric travel patterns are occurring, which cause bikes to accumulate at some stations while others become depleted. To fix this, we introduce a rebalancing algorithm. We will add a vehicle that periodically transfers bikes between stations.
Let $0 < \ell < h < 1$ be fixed thresholds. At time t, we classify stations as

$$\text{Under-supplied: } b_i(t) < \ell c_i, \tag{2}$$

$$\text{Over-supplied: } b_i(t) > h c_i. \tag{3}$$

Every τ minutes, up to M bikes are moved from over-supplied stations to under-supplied stations, subject to feasibility ($b_i(t) \geq 1$ at donors and $b_i(t) \leq c_i - 1$ at receivers). Receivers are prioritised by highest demand intensity λ_i, so that bikes are moved preferentially to stations where shortages are most costly. Figure 3 illustrates the resulting rebalancing activity over time.

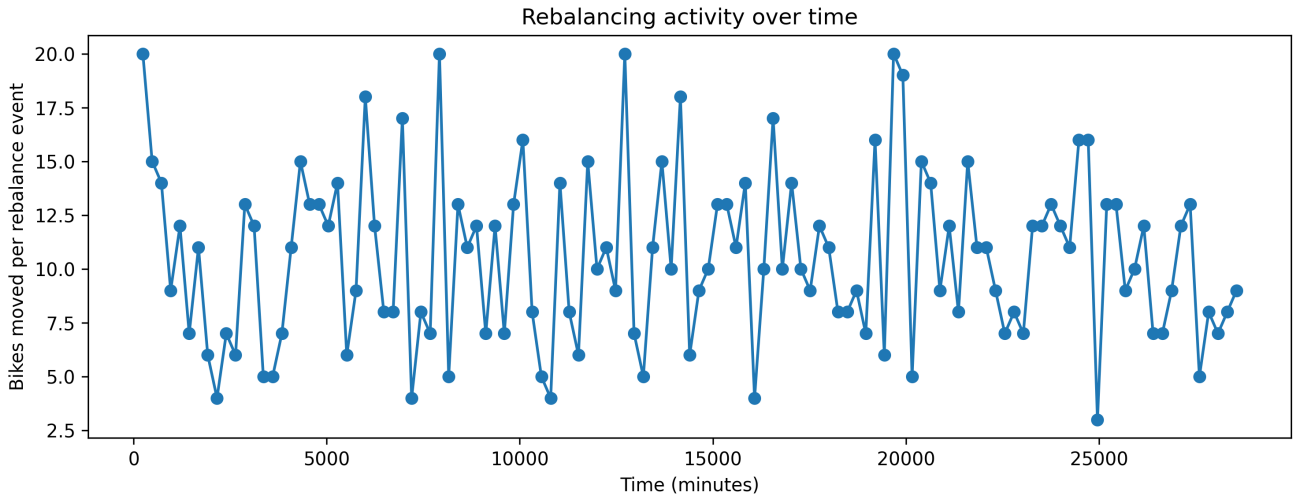


Figure 3: Bikes moved during each rebalancing event over time, with rebalancing events occurring every $\tau = 240$ minutes with a maximum of $M = 20$ bike moves per event (the size of the van/trailer), and thresholds $\ell = 0.25$ and $h = 0.50$. Variation reflects changes in shortages and surpluses driven by stochastic demand.

4.2 Optimisation Results

To evaluate the impact of each of these optimisations, we compare three scenarios:

1. Baseline: uniform dock capacity at each station with zero optimisation,
2. Capacity only: station-specific dock capacities c_i from (F),
3. Capacity + rebalancing: station-specific capacities with periodic bike transfers.

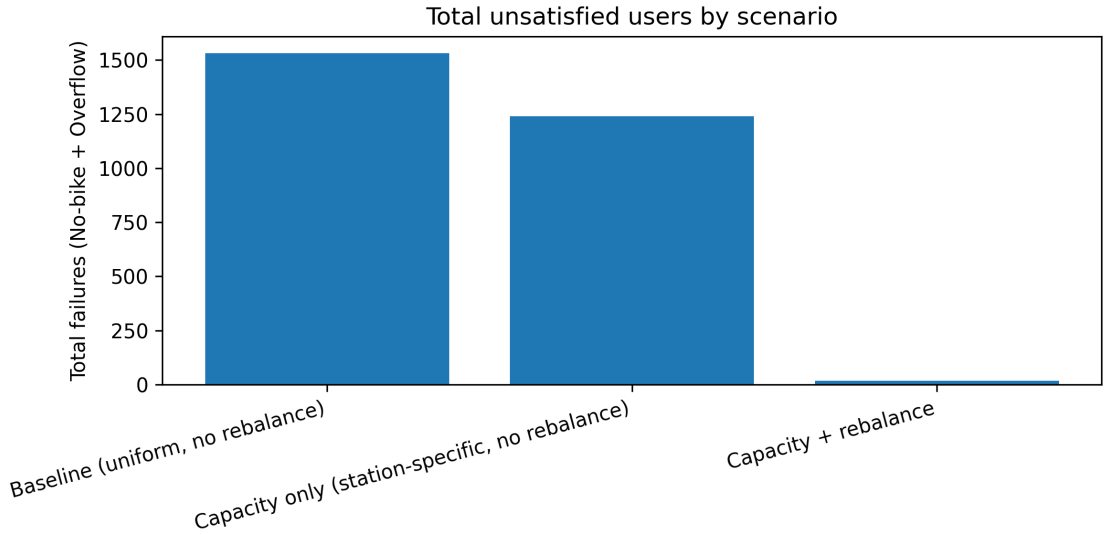


Figure 4: Total failures (objective J) across scenarios: baseline, station-specific capacities, and station-specific capacities (with the same conditions used previously)

Figure 4 shows that allocating station-specific dock capacities reduces total unsatisfied demand compared to the baseline, answering question (2).Results indicate that this reduces overflow errors, as bikes have more docks to enter. No-bike errors remain, however the rebalancing step eliminates nearly all of them along with the remaining overflow errors. This model identifies appropriate responses to sudden increases in demand or a need for more stations. Such actions could mean that we may need another van to be sent (reducing τ in the simulation) or increasing funding for more docks (increasing C_{tot}).

5 Extension: Mapping Intra-day Changes

From our original full scale model (2), we can see discrepancies between the pick-ups and returns for each day. We also observe in our rebalancing that the number of bikes moved by our vehicle varies significantly (3). To understand these observations further, we can map the state changes of the docks throughout the day using proportional symbol mapping.
Proportional Symbol Mapping utilises thematic maps to display quantitative data using symbols whose variance in size and shape represents quantities in specific geographical locations[2]

5.1 Model

To visualise station behaviour over a fixed time window, we use the percentage of time each station is empty or full, following approaches used in prior analyses of the Belfast scheme [1]. The following graph plots the percentage of time, over our designated time period, that each station is either empty or overloaded.

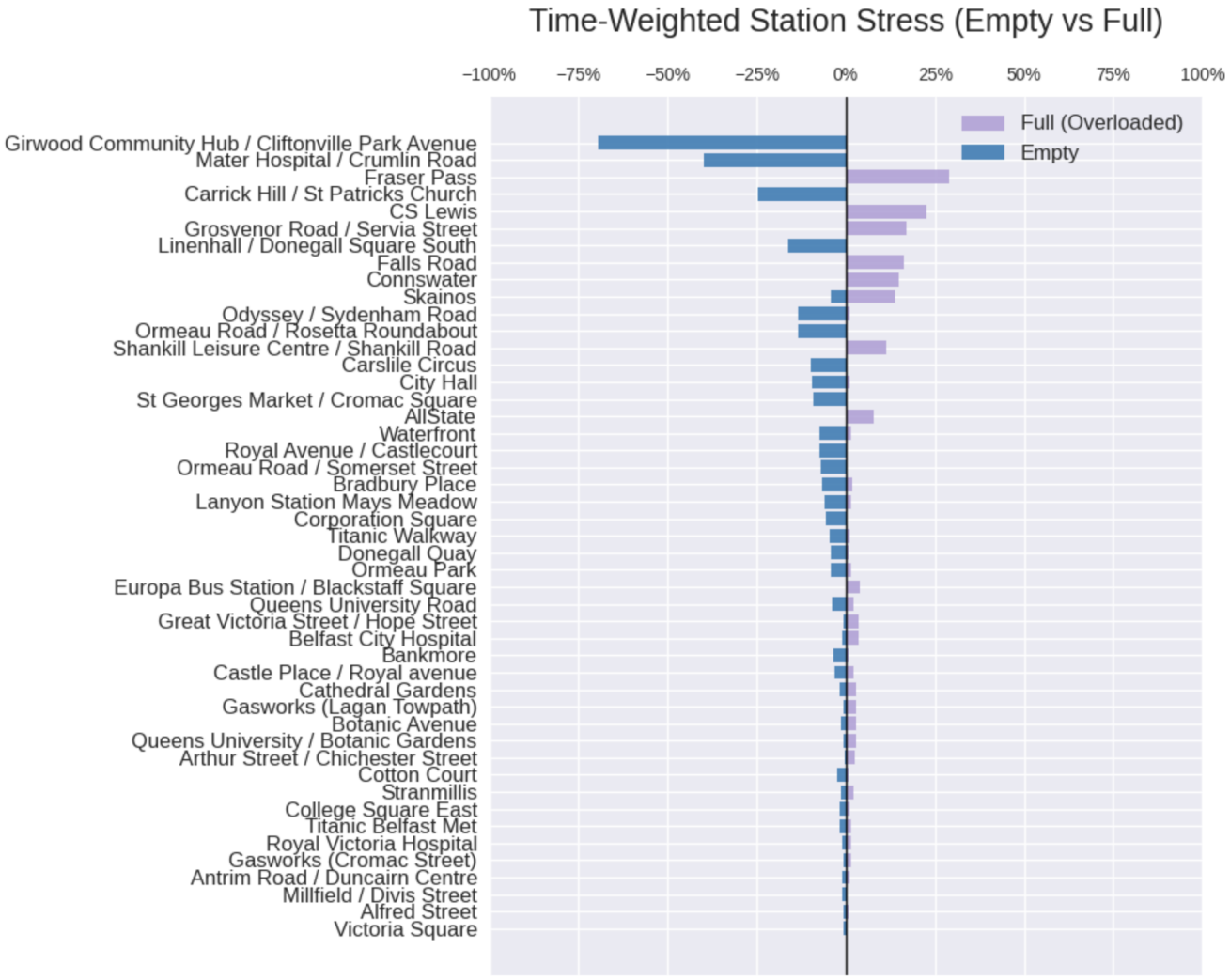
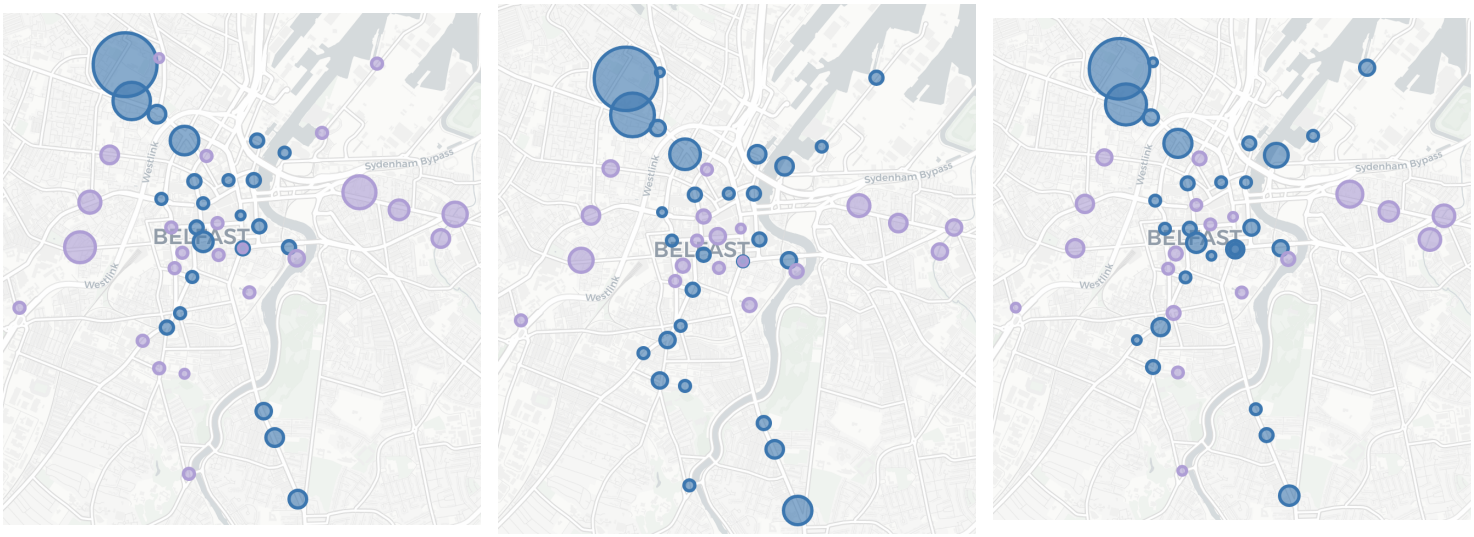


Figure 5: A diverging bar chart showing the percentage time from 1st April to 31st October that each station has spent either empty (blue bars) or full (purple bars)

The graph shows that most stations experience little stress, spending under 10% of the time either empty or full. A few stations stand out as problematic: Girwood Community Hub and Mater Hospital are frequently empty, while Fraser Pass, Grosvenor Road, and Falls Road are often at capacity. Using these results, we construct proportional-symbol maps for three time windows (09:00–11:00, 15:00–17:00, and 19:00–21:00). Circle size reflects how often a station is empty or overloaded, and colour indicates whether it tends to be empty or full

[2]For an accessible overview of proportional symbol maps, see `https://atlas.co/blog/what-is-a-proportional-symbol-map/`



(a) 09:00–11:00 (b) 15:00–17:00 (c) 19:00–21:00

Figure 6: Purple circles represent full stations and blue circles represent empty stations. Circle size is proportional to the duration spent in each state.

We can compare these figures to make the following inferences. We now examine question (3) regarding the underlying factors driving the differences in pickup and re:

- **Morning Inflow Pattern.** Comparing the morning and afternoon figures shows strong movement *toward* the city centre as people cycle in for work or study, leaving several outer stations depleted and many central stations overloaded. This can be seen by both the south and the north turning from purple (full) to blue (empty)
- **Evening Redistribution.** After the workday, travel shifts outward as people return to residential areas. This produces a broader redistribution across the network, but not a complete correction: some central stations remain overused while certain peripheral stations continue to experience shortages.

6 Conclusions, Limitations and Further Work

This project models a stochastic simulation of the Belfast Bike-Share system to understand station imbalance, assess optimisation strategies, and examine how demand patterns vary throughout the day.

Conclusions

- **What issues cause uneven pickup and return rates?** From (2), residential and student stations act as net sources of bikes, while employment and commercial stations act as net sinks. This structural imbalance produces the main failures in the system: no-bike shortages at origins and overflow at central destinations.
- **How can the system be optimised?** From (4), the optimisation results show that the most cost-efficient configuration modelled is a single operator rebalancing bikes by van every four hours. Station-specific dock capacities reduce overflow but do not resolve shortages at high-demand origins, whereas periodic rebalancing sharply decreases total failures without requiring additional bikes or infrastructure.
- **What factors drive these differences?** The intra-day mapping reveals strong time-dependent flows, with a clear movement from the outskirts of Belfast toward the city centre between 11:00 and 15:00. This incomplete return flow later in the day leaves persistent surpluses and deficits, showing that temporal demand patterns—not just station location—drive the uneven pickup and return rates.

6.1 Limitations

The model is based on historical trip data and therefore assumes future demand patterns remain consistent, neglecting the effects of weather, special events and behavioural change. Dock capacities are fixed, and rebalancing is rule based, without accounting for operational constraints such as vehicle routing or staffing. At most one bike may depart from any station per minute, While multiple bikes may arrive simultaneously if they come from different stations. This may underestimate congestion and no-bike failures at high-demand stations

6.2 Further Work

Future extensions could incorporate stochastic travel times, allowing bikes to remain in transit between stations, and time-varying departure rates to better capture peak demand. Explicit modelling of rebalancing costs would also enable trade-off analysis between infrastructure investment, operational effort, and service quality.

References

[1] N. Demidova, A. Novakovic, and A. H. Marshall, *Optimizing the Belfast Bike Sharing Scheme*, in Intelligent Systems and Applications (IntelliSys 2021), Lecture Notes in Networks and Systems, vol. 295, Springer, 2022.

[2] L. Zhang, Z. Xiao, S. Ren, Z. Qin, R. S. M. Goh, and J. Song, *Measuring the vulnerability of bike-sharing system*, Transportation Research Part A: Policy and Practice, vol. 163, pp. 353–369, 2022.

[3] X. Qian, R. (Ivan) Xiao, R. Joby, and M. Jaller, *Optimizing bikeshare service to connect affordable housing units with transit service*, Journal of Transport Geography, vol. 117, 2024, article 103894.

[4] Queen's University Belfast, *Number of students on campus*, https://www.qub.ac.uk/Study/

[5] Belfast Met Titanic Quarter Campus, *Number of student enrolments*, https://www.accessable.co.uk/belfast-metropolitan-college/titanic-quarter-campus

[6] F. Soriguera, V. Casado, and E. Jiménez, *A simulation model for public bike-sharing systems*, Transportation Research Procedia, 2018. https://www.sciencedirect.com/science/article/pii/S2352146518302412

[7] V. J. Løwe, *What is a proportional symbol map?* https://atlas.co/blog/what-is-a-proportional-symbol-map/

[8] V. J. Løwe,

[9] R. Barvir, M. Holub, and A. Vondrakova, *Proportional Symbol Maps: Value-Scale Types, Online Value-Scale Generator and User Perspectives*, ISPRS International Journal of Geo-Information, 2025. https://www.mdpi.com/2220-9964/14/9/340

A Appendix: Raw Data Description

The raw Belfast Bikes datasets were provided as multiple Excel datasheets, each corresponding to a different month of recorded journeys. Each worksheet contained one row per completed journey, including identifiers for the start station and end station.

id	city_id	bike_id	start_date	start_place_id	start_place_city_id	start_place_name	start_place_station_number	start_place_lat	start_place_lng	start_channel_id
115275307	238	127486	5/1/2021 0:04	20753840	238	Botanic Avenue	3909	54.58932893	-5.93350768	264
115275330	238	127708	5/1/2021 0:05	20753840	238	Botanic Avenue	3909	54.58932893	-5.93350768	296
115275370	238	127569	5/1/2021 0:07	20754767	238	City Hall	3902	54.59717668	-5.93082547	264
115275373	238	127618	5/1/2021 0:07	20754767	238	City Hall	3902	54.59717668	-5.93082547	296
115275424	238	127674	5/1/2021 0:09	20753582	238	Bankmore	3918	54.59267956	-5.93152821	264
115275444	238	127397	5/1/2021 0:11	20753582	238	Bankmore	3918	54.59267956	-5.93152821	296
115275450	238	127648	5/1/2021 0:12	20753582	238	Bankmore	3918	54.59267956	-5.93152821	264
115275507	238	127536	5/1/2021 0:15	20754767	238	City Hall	3902	54.59717668	-5.93082547	296
115275520	238	127478	5/1/2021 0:16	20756307	238	Queens University / Botanic Gardens	3934	54.58390144	-5.93281031	296
115275521	238	127470	5/1/2021 0:16	20756307	238	Queens University / Botanic Gardens	3934	54.58390144	-5.93281031	296
115275523	238	127491	5/1/2021 0:16	20756307	238	Queens University / Botanic Gardens	3934	54.58390144	-5.93281031	296

end_date	end_place_id	end_place_city_id	end_channel_id	end_place_name	end_place_station_number	end_place_lat	end_place_lng	duration_sec	invalid	price
5/1/2021 0:15	26211679	238	37	Belfast City Hospital	3935	54.58684841	-5.93933344	646	0	1
5/1/2021 0:14	26211679	238	37	Belfast City Hospital	3935	54.58684841	-5.93933344	536	0	1
5/1/2021 0:15	26211679	238	37	Belfast City Hospital	3935	54.58684841	-5.93933344	515	0	1
5/1/2021 0:15	26211679	238	37	Belfast City Hospital	3935	54.58684841	-5.93933344	518	0	1
5/1/2021 0:37	20755892	238	37	Great Victoria Street / Hope Street	3911	54.593469	-5.9344089	1630	0	0
5/1/2021 0:20	20753582	238	37	Bankmore	3918	54.59267956	-5.93152821	560	0	1
5/1/2021 0:14	20753582	238	37	Bankmore	3918	54.59267956	-5.93152821	166	1	1
5/1/2021 0:27	20754767	238	37	City Hall	3902	54.59717668	-5.93082547	709	0	1
5/1/2021 0:28	20756307	238	37	Queens University / Botanic Gardens	3934	54.58390144	-5.93281031	736	0	1
5/1/2021 8:54	20756307	238	52	Queens University / Botanic Gardens	3934	54.58390144	-5.93281031	31067	0	10
5/1/2021 0:26	20756307	238	37	Queens University / Botanic Gardens	3934	54.58390144	-5.93281031	617	0	1

Figure 7: 12 observations from the May datasheet showing the range of raw data available to us.

B Appendix: Excel Processing Workflow

All monthly datasets were merged into a single dataset in order to maximise the number of observed journeys and reduce short-term or seasonal effects. This ensured that the movement patterns used in the model were based on the widest possible data sample.

id	city_id	bike_id	start_date	start_place_id	start_place_city_id	start_place_name	start_place_station_number	start_place_lat	start_place_lng	start_channel_id
130862360	238	127598	10/12/2021 14:01	20751801	238	Alfred Street	3923	54.59463449	-5.92744052	296
121548821	238	127470	7/8/2021 9:46	20751801	238	Alfred Street	3923	54.59463449	-5.92744052	296
118995547	238	127536	6/14/2021 17:15	20751801	238	Alfred Street	3923	54.59463449	-5.92744052	296
120606285	238	127472	6/29/2021 9:46	20751801	238	Alfred Street	3923	54.59463449	-5.92744052	296
124770620	238	127340	8/9/2021 0:06	20751801	238	Alfred Street	3923	54.59463449	-5.92744052	296
115349249	238	127659	5/2/2021 19:18	20751801	238	Alfred Street	3923	54.59463449	-5.92744052	296
123950892	238	127356	7/31/2021 0:07	20751801	238	Alfred Street	3923	54.59463449	-5.92744052	296
128041778	238	127715	9/11/2021 15:07	20751801	238	Alfred Street	3923	54.59463449	-5.92744052	296
116770381	238	127649	5/22/2021 15:09	20751801	238	Alfred Street	3923	54.59463449	-5.92744052	264
118751306	238	127567	6/11/2021 21:39	20751801	238	Alfred Street	3923	54.59463449	-5.92744052	296
122211786	238	127410	7/14/2021 22:31	20751801	238	Alfred Street	3923	54.59463449	-5.92744052	296

end_date	end_place_id	end_place_city_id	end_channel_id	end_place_name	end_place_station_number	end_place_lat	end_place_lng	duration_sec	invalid	price
10/14/2021 13:38	20751801	238	37	Alfred Street	3923	54.59463449	-5.92744052	171392	0	20
7/9/2021 9:00	20756085	238	37	Odyssey / Sydenham Road	3921	54.60392299	-5.91724277	83679	0	10
6/15/2021 16:09	20755535	238	15	Fraser Pass	3947	54.60033711	-5.90551615	82421	0	10
6/30/2021 8:27	20756347	238	37	Queens University Road	3933	54.5843803	-5.9367269	81657	0	10
8/9/2021 22:06	20754188	238	15	CS Lewis	3944	54.59827991	-5.89057088	79190	0	10
5/3/2021 16:20	20754700	238	37	Lanyon Station Mays Meadow	3905	54.59532133	-5.91645956	75745	0	10
7/31/2021 20:47	26211679	238	37	Belfast City Hospital	3935	54.58684841	-5.93933344	74410	0	10
9/12/2021 10:37	20751801	238	52	Alfred Street	3923	54.59463449	-5.92744052	70168	0	10
5/23/2021 10:17	20754767	238	37	City Hall	3902	54.59717668	-5.93082547	68869	0	10
6/12/2021 14:15	20844102	238	15	Ormeau Road / Somerset Street	3913	54.5781418	-5.9187871	59763	0	10
7/15/2021 14:30	20751801	238	15	Alfred Street	3923	54.59463449	-5.92744052	57549	1	10

Figure 8: 12 observations from the combined datasheet, sorted by starting station to display the range of starting dates we use in our model.

Journeys containing missing or invalid station identifiers (e.g. unknown stations, damaged or lost bikes) were removed. These observations cannot be represented as valid ordered pairs $(i, j) \in \mathcal{S} \times \mathcal{S}$.

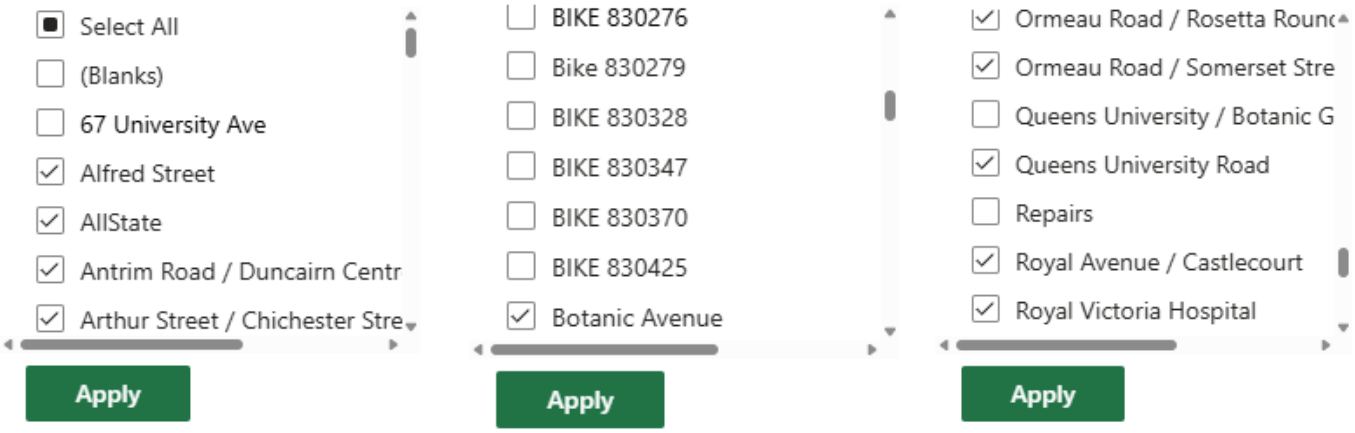


Figure 9: These filters display what we removed from the combined data sheet, these include missing data, invalid stations, duplicate stations

After the cleaning of raw data, the number of journeys between each pair of stations was counted to form the flow matrix used in the simulation.

C Appendix: Data Cleaning and Preprocessing

In order to transform the data into something we can model and simulate, we first have to clean the data.

To obtain a complete representation of user behaviour over the full period of time, all monthly datasets were merged into a single dataset.

Combining the datasets ensures that the resulting model uses the maximum amount of data to observe a wider range of usage patterns. This eliminates short term or seasonal effects present in individual months.

During the merging process, some journeys were found to contain missing or invalid station identifiers, such as journeys with an unknown start or end station, or damaged or lost bikes. These journeys cannot be represented as ordered pairs $(i, j) \in \mathcal{S} \times \mathcal{S}$ and were therefore removed from the dataset.

This cleaning step ensures that every remaining journey in the data is a valid transition between two stations in the network and that all stations are well-defined.

D Appendix: Construction of Flow and Transition Matrices

Let $\mathcal{S} = \{1, 2, \ldots, N\}$ denote the set of all stations included in the newly cleaned dataset, and let each recorded journey be an ordered pair $(i, j) \in \mathcal{S} \times \mathcal{S}$, where station i is the starting station and station j is the ending station.

Flow matrix

The flow matrix $F \in \mathbb{R}^{N \times N}$ is defined by

$$F_{ij} = \text{the number of journeys that start at station } i \text{ and end at station } j.$$

Each row of F corresponds to a starting station, and each column corresponds to an ending station.This means that the row sum

$$\sum_{j=1}^{N} F_{ij}$$

is the total number of trips starting at station i during the observation window.

Only journeys with valid start and end stations were included in F, ensuring that every entry in F is a valid ordered pair.

Transition probability matrix

To model bike movements, we need probabilities, so the flow matrix is row-normalised, which produces a transition probability matrix $P \in \mathbb{R}^{N \times N}$, with entries

$$P_{ij} = \begin{cases} \dfrac{F_{ij}}{\sum_{k=1}^{N} F_{ik}}, & \text{if } \sum_{k=1}^{N} F_{ik} > 0, \\ 0, & \text{otherwise.} \end{cases}$$

Each row of P is a probability distribution over ending stations. It also means that

$$\sum_{j=1}^{N} P_{ij} = 1 \quad \text{for all stations with observed departures.}$$

E Appendix: Stochastic Departure Model

Trips departing from station i are modelled using a Bernoulli approximation to a Poisson process with rate λ_i. In each simulated minute, a random variable

$$s_i \sim \text{Uniform}(0, 1)$$

is generated. A departure from station i occurs in that minute if $s_i < \lambda_i$, provided that $b_i(t) > 0$.

When a departure occurs, the destination station j is selected according to the transition probabilities P_{ij}.

F Appendix: Simulation Initialisation Details

Let $b_i(t)$ be the number of bikes present at station i at time t. The total number of bikes in the system, b_{tot}, is a constant set throughout the simulation.

We initially distribute bikes using a demand weighted stochastic allocation.

Let

$$T_i = \sum_{j=1}^{N} F_{i,j}$$

denote the total number of departures from station i in the observed data window of length $(t_2 - t_1)$ minutes. The corresponding departure rate is

$$\lambda_i = \frac{T_i}{t_2 - t_1}. \tag{4}$$

Normalised departure weights are then defined as

$$\pi_i = \frac{\lambda_i}{\sum_{k=1}^{N} \lambda_k}, \qquad \sum_{i=1}^{N} \pi_i = 1. \tag{5}$$

The initial bike configuration is generated using a multinomial allocation

$$(b_1(0), \ldots, b_N(0)) \sim \text{Multinomial}(b_{\text{tot}}; \pi_1, \ldots, \pi_N), \tag{6}$$

which ensures that $\mathbb{E}[b_i(0)] = b_{\text{tot}}\pi_i$. This allocates bikes proportionally to observed departure demand, reflecting the fact that no-bike failures occur at trip origins.

A total dock budget C_{tot} is also specified. The capacity of station i, denoted c_i, represents the maximum number of bikes that can be accommodated at that station. An overflow failure occurs when $b_i(t) = c_i$ and a bike attempts to arrive. In the baseline configuration, dock capacities are initially distributed uniformly,

$$c_1 = c_2 = \cdots = c_N = \frac{C_{\text{tot}}}{N}. \tag{7}$$

Finally, physical constraints are enforced so that $0 \le b_i(0) \le c_i$ for all stations. Any bikes allocated above capacity are removed and redistributed to stations with available free docks until all bikes are placed. This produces a feasible and realistic initial system state consistent with both demand and infrastructure constraints.

G Appendix: Python Implementation

All simulations were implemented in `Python`, using `NumPy` for sampling and operations, `Pandas` for Excel data handling, and `Matplotlib` for visualisation. The appendix below shows pseudocode that mirrors the final algorithms used to generate the results in this report.

G.1 Function overview

Table 1 summarises the main functions used in the simulations and optimisations.

Function	Purpose
`load_flow_matrix_from_sheet1`	Reads the Excel table, removes totals and invalid rows/columns, and makes the flow matrix F.
`build_markov_inputs`	Builds the transition matrix P and per-minute departure rates λ from F.
`initialise_bikes`	Generates the initial bike distribution.
`simulate_bike_share`	Main simulation of bike movements, failures, and rebalancing.
`propose_capacities`	Allocates dock capacities under a total dock budget.
`rebalance_bikes`	Moves bikes between stations to fix shortages and overflows during the simulation.

Table 1: Summary of the main Python functions used in the model implementation.

G.2 Excel parsing and construction of the flow matrix

The journey data are provided as an Excel pivot table.

Pseudocode: `load_flow_matrix_from_sheet1`

```
Read entire worksheet with no headers
Extract destination station labels from header row
Trim trailing empty columns and remove "Grand Total"

Extract origin station labels from first column
Stop at row labelled "Grand Total"
Remove empty or invalid labels

Extract numeric block and coerce to numbers
Replace missing values with zero

Reindex onto union of row/column station names
Fill missing entries with zero

Drop stations with zero total activity
Return flow matrix F
```

G.3 Construction of Markov inputs

From the flow matrix F, the transition matrix P is created with departure rates λ_i. The time window is converted into minutes.
Pseudocode: `build_markov_inputs`

```
F <- numeric matrix from DataFrame
row_totals[i] = sum_j F[i,j]

minutes = length of data window in minutes
lambda[i] = row_totals[i] / minutes

for each station i:
    if row_totals[i] > 0:
        P[i,:] = F[i,:] / row_totals[i]
    else:
        P[i,i] = 1   # self-loop fallback

return stations, F, P, lambda
```

G.4 Initialisation with capacity constraints

Initial bike locations are generated using a multinomial distribution weighted by station demand.
Pseudocode: `initialise_bikes`

```
Normalise weights
bikes <- Multinomial(total_bikes, weights)

Clip bikes[i] to capacity[i]
leftover = total_bikes - sum(bikes)

while leftover > 0 and free capacity exists:
    choose station k proportional to free capacity
    bikes[k] += 1
    leftover -= 1

return bikes
```

G.5 Core simulation algorithm

The model progresses with one minute time steps. Each station generates at most one start attempt per minute using Bernoulli approximations with a probability λ_i. Destinations are sampled from the transition matrix P.
Pseudocode: `simulate_bike_share`

```
Initialise random number generator
Initialise bikes using initialise_bikes()

Initialise counters for:
    no-bike failures, overflow failures
    attempted starts, successful starts, successful ends
    successful OD flows

for each minute t:
    arrivals[i] = 1 if Uniform(0,1) < lambda[i]

    for each station i with arrivals[i] = 1:
        attempted_starts[i] += 1

        if bikes[i] == 0:
            no-bike failure
            continue

        j ~ Categorical(P[i,:])

        if bikes[j] == capacity[j]:
            overflow failure
            continue

        bikes[i] -= 1
        bikes[j] += 1
        record successful trip

    record system state every fixed interval

    if rebalancing enabled and t is a rebalance time:
        move bikes using rebalance_bikes()

return final state, counters, and time series
```

G.6 Station-specific dock allocation

Dock capacities are allocated under a fixed total budget using weights determined by demand.
Pseudocode: `propose_capacities`

```
Compute weights w[i] from inbound demand
Normalise weights

Assign base capacity min_cap to all stations
Distribute remaining capacity proportional to w

If rounding error exists:
    add/remove single docks until total budget satisfied

return capacity vector
```

G.7 Active bike rebalancing

Rebalancing is applied periodically. Stations below a lower threshold are treated as receivers, while stations above an upper threshold act as donors. **Pseudocode:** `rebalance_bikes`

```
Identify receivers: bikes < low_frac * capacity
Identify donors:    bikes > high_frac * capacity

Sort receivers by decreasing lambda
Sort donors by decreasing surplus

while moves < max_moves and donors/receivers remain:
    move one bike from donor to receiver
    update donor/receiver status

return number of bikes moved
```

G.8 Extension Diverging Bar Chart

This step constructs the diverging bar chart by ordering stations by their stress level and plotting the proportion of time each one spends empty or full. **Pseudocode:** `create_stress_chart`

```
Sort stations by magnitude of stress
Reorder empty and full percentages accordingly
Convert empty percentages to negative values

Create horizontal bar chart:
    plot full percentages to the right
    plot empty percentages to the left
    add central vertical axis at zero
    label stations on the y-axis
    format x-axis as percentages
    set limits from -100 to 100

Add title and layout adjustments
Display chart
```

G.9 Extension Creation

This step applies the time-window model to three key periods of the day, allowing differences in empty and full station behaviour to be visualised spatially across Belfast **Pseudocode:** `generate_timewindow_maps`

```
For each chosen time window:
    run compute_timewindow to obtain:
        percent_empty, percent_full, magnitude

    create a new map centred on Belfast

    for each station:
        read coordinates
        choose colour based on empty vs full
        set marker radius from magnitude
        add circle marker with tooltip

    save map for that time window
```

G.10 Reproducibility and assumptions

All stochastic components use a fixed random seed via `numpy.random.default_rng(seed)` to ensure reproducibility. Trips are assumed to complete within a single time step, meaning travel time is neglected and bikes are transferred immediately once a trip is successful.

H Appendix: Supplementary Figures and Outputs

Now that we have a model, we would like to validate it's results. We can accomplish this by checking that the number of bikes stays constant for the duration of the day to ensure that no bikes are lost or any unexpected behaviour occurs throughout the model run.

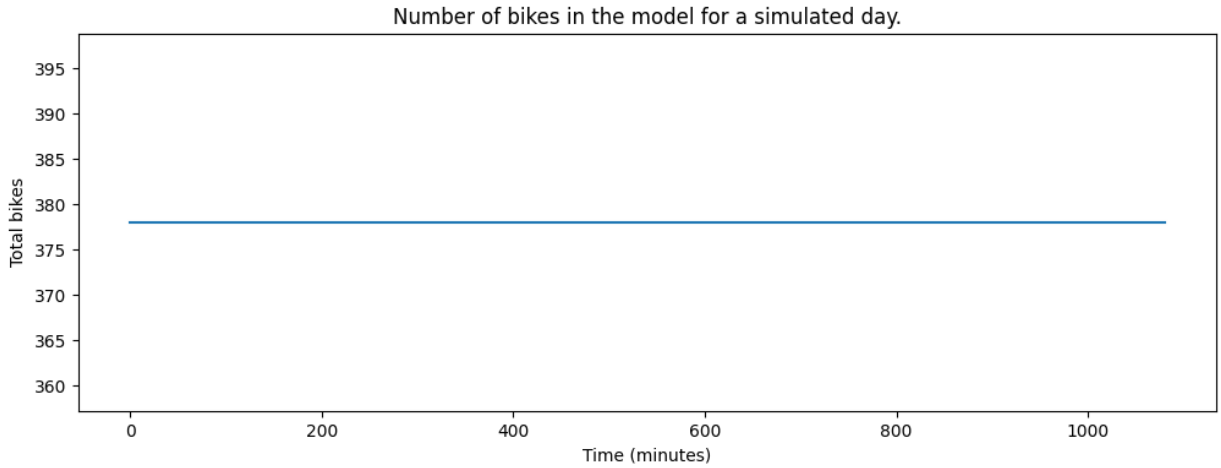


Figure 10: Number of bikes in model for a simulated day. We see a horizontal unchanging line at $y = 378$. This is the number of bikes we started with and as seen it doesn't change between $t = 0$ and $t = 1440$ hence we can say that no bikes are lost during the simulated day.

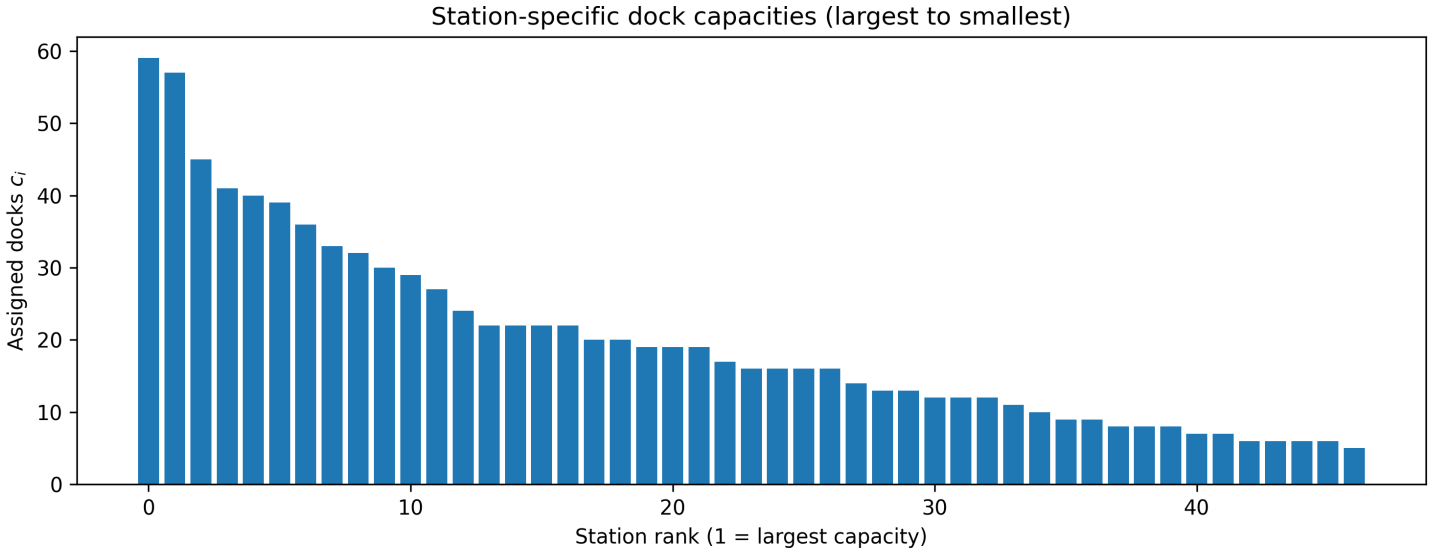


Figure 11: Station-specific dock capacities c_i produced by the allocation rule (**??**), with $c_{\min} = 5$, $C_{\text{tot}} = 940$ and using demand weights. Stations are ordered from largest to smallest assigned capacity.

Figure 11 shows that allocation naturally creates "large" stations and "small" stations, reflecting observed usage patterns from our data. This reduces the likelihood that busy destinations reach capacity.